Task 18:

Write unit test cases for microservices using a testing framework. Test service logic, REST controllers, and data handling independently to ensure reliability.

Aim

To write **unit test cases** for microservices using a testing framework to test **service logic, REST controllers, and data handling independently**, ensuring application reliability.

Algorithm

1. Add testing dependencies (JUnit, Mockito, Spring Boot Test).
2. Create test classes for:
    - Service Layer
    - Controller Layer
3. Use **Mockito** to mock repository dependencies.
4. Write test cases for:
    - Fetch data
    - Save data
    - Update data
    - Delete data
5. Use **MockMvc** for controller testing.
6. Run tests using JUnit.
7. Verify outputs and assertions.

Code Implementation

1. Dependencies (pom.xml)

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-test</artifactId>
   <scope>test</scope>
</dependency>
```

2. Service Layer Testing

ProductServiceTest.java

```
package com.example.demo.service;

import com.example.demo.entity.Product;
import com.example.demo.repository.ProductRepository;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.springframework.boot.test.context.SpringBootTest;

import java.util.*;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.Mockito.*;

@SpringBootTest
public class ProductServiceTest {

    @Mock
    private ProductRepository repo;

    @InjectMocks
    private ProductService service;

    @Test
    public void testGetAllProducts() {
        List<Product> list = Arrays.asList(
                new Product("Laptop", 50000, 2),
                new Product("Mobile", 20000, 5)
        );

        when(repo.findAll()).thenReturn(list);

        List<Product> result = service.getAllProducts();

        assertEquals(2, result.size());
    }
```

```
    @Test
    public void testSaveProduct() {
        Product p = new Product("Tablet", 15000, 3);

        when(repo.save(p)).thenReturn(p);

        Product result = service.saveProduct(p);

        assertEquals("Tablet", result.getName());
    }
}
```

3. Controller Testing

ProductControllerTest.java

```
package com.example.demo.controller;

import com.example.demo.entity.Product;
import com.example.demo.service.ProductService;
import org.junit.jupiter.api.Test;
import org.mockito.Mockito;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Arrays;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(ProductController.class)
public class ProductControllerTest {

    @Autowired
    private MockMvc mockMvc;
```

```
    @MockBean
    private ProductService service;

    @Test
    public void testGetAll() throws Exception {

        Mockito.when(service.getAllProducts())
                .thenReturn(Arrays.asList(new Product("Laptop", 50000, 2)));

        mockMvc.perform(get("/products"))
                .andExpect(status().isOk())
                .andExpect(jsonPath("$[0].name").value("Laptop"));
    }
}
```

4. Repository Testing (Optional)

```
@DataJpaTest
public class ProductRepositoryTest {

    @Autowired
    private ProductRepository repo;

    @Test
    public void testSave() {
        Product p = new Product("Watch", 2000, 4);
        Product saved = repo.save(p);

        assertNotNull(saved.getId());
    }
}
```

Output:

src > test > java > com > demo > customer > controller > CustomerControllerTest.java > ...

/tests write unit test for customer controller class for create and get customer by id

Accept Discard 1 change

```java
package com.demo.customer.controller;

import com.demo.customer.model.Customer;
import com.demo.customer.service.CustomerService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.mockito.Mockito.*;

class CustomerControllerTest {

    @Mock
    private CustomerService customerService;

    private CustomerController customerController;

    @BeforeEach
    void setUp() {
        MockitoAnnotations.openMocks(this);
        customerController = new CustomerController(customerService);
    }

    @Test
    void createCustomer_shouldReturnCreatedCustomer() {
        // Arrange
        Customer customer = new Customer();
        when(customerService.createCustomer(customer)).thenReturn(customer);

        // Act
        ResponseEntity<Customer> response = customerController.createCustomer(customer);

        // Assert
        assertEquals(HttpStatus.CREATED, response.getStatusCode());
        assertEquals(customer, response.getBody());
        verify(customerService, times(1)).createCustomer(customer);
    }

    @Test
    void getCustomer_shouldReturnExistingCustomer() {
```

Result

Unit tests successfully implemented Service logic tested independently Controller APIs validated using MockMvc Application reliability improved ✅